Práctica 1
Tecnologías de Registro Distribuido y Blockchain (BC)

Álvaro Fernández Carrillo

1. Análisis y Definición del Escenario....2
1.1 Contexto y Problemática (Evolución desde la Práctica 1).... 2
1.2 Justificación de la Solución Híbrida (IPFS + Ethereum).... 2
2. Diseño de la Arquitectura y Casos de Uso.... 2
2.1 Esquema de Comunicaciones del Sistema....2
2.2 Actores y Funcionalidades....2
3. Implementación Técnica....3
3.1 Smart Contract (AcademicCertificatesIPFS.sol).... 3
3.2 Aplicación Web en React (App.js).... 3
4. Secuencia de Pruebas y Validación....3
5. Lecciones Aprendidas e Incidencias Resueltas....4
1. Conflicto de Puertos y Sintaxis en PowerShell (Docker Windows)....4
2. Restricción de Origen en Navegadores.... 4
3. Configuración de Cabeceras CORS en el Nodo IPFS....4



1. Análisis y Definición del Escenario

1.1 Contexto y Problemática (Evolución desde la Práctica 1)

En la Práctica 1 se diseñó e implementó el contrato inteligente ***AcademicCertificates.sol***, permitiendo registrar la validez de un título académico en Ethereum mediante variables de texto simple (nombre del alumno y carrera). Sin embargo, este enfoque presentaba una limitación clave para su adopción en un entorno real, es necesario acceder al **documento físico o PDF oficial firmado digitalmente**, y no únicamente a una confirmación textual en la blockchain.

Almacenar archivos digitales pesados (como títulos en formato PDF o escaneos en alta resolución) directamente en el almacenamiento permanente (*storage*) de la Máquina Virtual de Ethereum es técnicamente inviable debido a los astronómicos costes de *gas* asociados (20. 000 unidades de gas por cada palabra de 32 bytes escrita en *storage*).

1.2 Justificación de la Solución Híbrida (IPFS + Ethereum)

Para solventar esta limitación, el Ejercicio 2 expande el sistema implementando una **arquitectura de almacenamiento híbrida descentralizada**:

1. **Almacenamiento Descentralizado de Contenidos (IPFS):** El archivo PDF original del título se almacena en IPFS. IPFS devuelve un Identificador de Contenido único criptográfico (*CID*, basado en el hash del archivo). Si el contenido del PDF se altera aunque sea en un solo bit, su CID cambia por completo.
2. **Inmutabilidad y Sellado en Blockchain (Ethereum):** El contrato inteligente almacena únicamente los metadatos esenciales del alumno junto con el **CID de IPFS** (32 - 46 caracteres de texto), reduciendo el consumo de gas al mínimo.
3. **Verificación Abierta y Gratuita:** Cualquier verificador puede consultar el contrato en la red Ethereum de forma gratuita (función de lectura *view*), verificar el estado de validez del título (*isValid == true*) y recuperar el CID para descargar el PDF original directamente desde cualquier nodo o gateway de IPFS.

2. Diseño de la Arquitectura y Casos de Uso

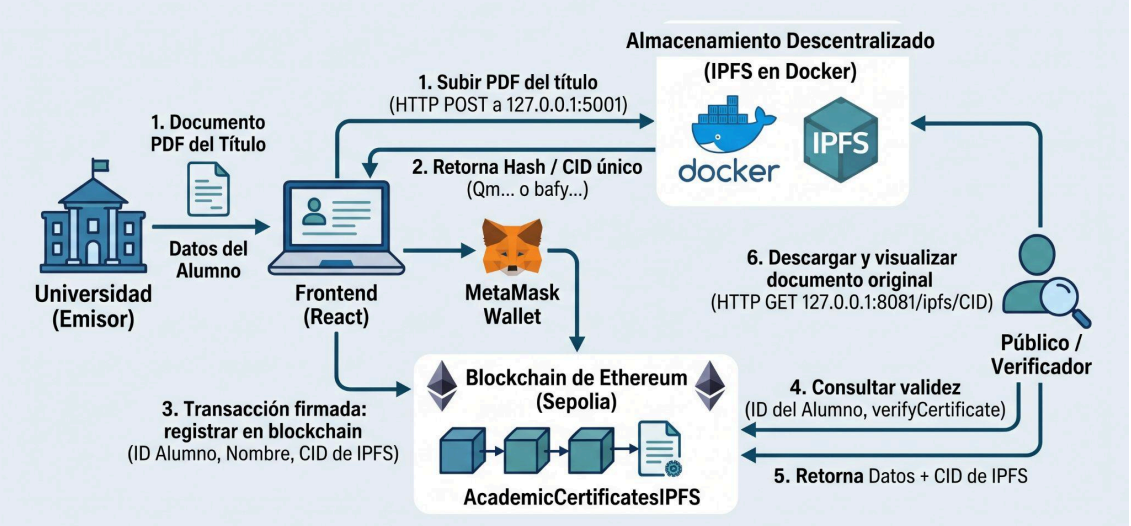
2.1 Esquema de Comunicaciones del Sistema

La arquitectura desarrollada conecta el navegador web (React), la cartera del usuario (MetaMask), el nodo local de almacenamiento P2P (IPFS via Docker) y la red de bloques (Ethereum/Sepolia).

2.2 Actores y Funcionalidades

1. **Universidad / Emisor (*universityOwner*):**
    - Selecciona el archivo PDF correspondiente al título oficial.
    - La DApp sube el archivo al nodo IPFS y obtiene su hash (CID).
    - Ejecuta la transacción firmada en MetaMask hacia el Smart Contract para asociar el identificador del certificado con el CID de IPFS.
2. **Verificador Público (Empleador / Alumno / Tercero):**
    - Introduce el identificador del certificado (ej. DNI o ID único) en el portal de verificación.

- Obtiene instantáneamente el estado de validez, la fecha de emisión y el enlace de descarga directa del documento desde la red IPFS.



3. Implementación Técnica

3.1 Smart Contract (*AcademicCertificatesIPFS.sol*)

Disponible en **AcademicCertificatesIPFS.sol**

3.2 Aplicación Web en React (*App.js*)

Disponible en **dapp_ipfs/src/App.js**

4. Secuencia de Pruebas y Validación

El sistema completo fue probado verificando la integración de todos sus componentes:

1. **Despliegue del Smart Contract:**
   - El contrato *AcademicCertificatesIPFS.sol* fue compilado con Solidity 0.8.30 y desplegado en la red (Remix VM). La dirección generada fue asignada al archivo *src/contracts/src/addresses.js.*
2. **Arranque del Nodo IPFS:**
   - Se inició el contenedor Docker *ipfs_host* mapeando los puertos de red y aplicando los permisos CORS necesarios.
3. **Prueba de Emisión de Título:**
   - Se introdujeron los datos de prueba (ID: 12345678X, Alumno: Álvaro Fernández, Carrera: Máster en Ciberseguridad) y se adjuntó un archivo de prueba.
   - El archivo fue añadido con éxito al nodo IPFS local, generando un CID válido (ej. bafybeib...).
   - MetaMask solicitó confirmación de la transacción para ejecutar la función *issueCertificate*. Tras la confirmación, la transacción se minó correctamente.



4. **Prueba de Verificación e Inmutabilidad:**
    - Introduciendo la clave 12345678X en el Verificador Público, la DApp retornó el estado “--- VÁLIDO ---”, junto con la fecha de minado y los datos correspondientes.
    - Al pulsar sobre "Ver / Descargar PDF Oficial", el navegador abrió la pasarela local de IPFS (http://127.0.0.1:8081/ipfs/<CID>), mostrando el documento idéntico al subido originalmente.



5. Lecciones Aprendidas e Incidencias Resueltas

Durante la realización de la práctica se presentaron diversos problemas técnicos de configuración e integración, los cuales se detallan a continuación junto con sus respectivas soluciones:

1. Conflicto de Puertos y Sintaxis en PowerShell (Docker Windows)

- Incidencia**:** Al ejecutar el comando docker run para lanzar el contenedor de IPFS en Windows PowerShell, la sintaxis original de bash *$PWD:/export* generó un error de interpretación (*InvalidVariableReferenceWithDrive*). Adicionalmente, el puerto host 8080 devolvió un error de binding (*bind: Only one usage of each socket address*) debido a que ya estaba ocupado por otro servicio del sistema operativo.

- **Solución:** Se adaptó el comando para PowerShell envolviendo la variable entre comillas dobles y corchetes "${PWD}:/export", y se modificó el mapeo del puerto del gateway web al puerto local 8081 (-p 127.0.0.1:8081:8080).

2. Restricción de Origen en Navegadores

- **Incidencia:** Al intentar subir archivos desde la aplicación en React, el navegador bloqueó la petición arrojando la excepción *TypeError: Failed to fetch y ERR_ADDRESS_INVALID*. Esto ocurre porque los navegadores modernos restringen el uso de la dirección 0.0.0.0 como destino en peticiones de cliente por motivos de seguridad.
- **Solución:** Se actualizó la cadena de conexión en el cliente de Kubo en React apuntando explícitamente a la interfaz de bucle de retorno local: *create('/ip4/127.0.0.1/tcp/5001')*.

3. Configuración de Cabeceras CORS en el Nodo IPFS

- **Incidencia:** La DApp web (ejecutada en *http://localhost:3000*) bloqueaba la carga de archivos debido a las políticas del mismo origen (Same-Origin Policy) del puerto API de IPFS (5001).
- **Solución:** Se aplicó la configuración de cabeceras de control de acceso desde la línea de comandos del contenedor escapando adecuadamente las comillas para PowerShell: *docker exec ipfs_host ipfs config --json API.HTTPHeaders.Access-Control-Allow-Origin '[\"http://0.0.0.0:5001\", \"http://localhost:3000\", \"http://127.0.0.1:5001\", \"https://webui.ipfs.io\"]'* Posteriormente se reinició el contenedor (*docker restart ipfs_host*).